

Private Federated Aggregation of LLM-Agent Memory

Usable Shared Memory with a Measurable Leakage-Drop Guarantee under Secure Aggregation and the Skellam Mechanism

Abstract

LLM agents increasingly accumulate **per-user memory**, distilled notes, preferences, and skill embeddings (as in A-MEM and Mem0), that make them personal and effective. Pooling these memories *across users* would let a fleet of agents share hard-won knowledge, but the memory objects are exactly the sensitive artifact: recent extraction (MEXTRA) and membership-inference (MRMMIA) attacks recover verbatim user facts from agent memory. We ask whether a shared memory pool can be made **useful and provably private at once**.

We aggregate per-user memory-note embeddings into a shared, bucketed **centroid memory** under **Secure Aggregation (SecAgg)** composed with the **discrete Skellam mechanism**, so the pooled memory carries a single central differential-privacy guarantee with **no trusted curator** and **no individual note in the clear**. The payload is new but the cryptographic path is verbatim prior work, which lets us reuse a rigorous discrete-DP accountant. We evaluate on **LongMemEval** (real long-horizon conversational memory) and on **LLM-distilled A-MEM/Mem0-style notes**, along two axes: (i) **retrieval utility** of the noised pool, and (ii) the **drop in worst-case re-identification** of vulnerable members.

Three findings.

(1) Usable DP memory. In the regime where agent payloads actually live, **tiny embedding dimension and coarse bucketing**, the private pool retains **95–96% of clean retrieval utility at $\epsilon \approx 9.3$** on real data.

(2) A measurable leakage-drop endpoint. Membership-inference AUC on the vulnerable low-count tail falls from **0.88–0.99 (near-certain) to ≈ 0.51 (chance) at $\epsilon \approx 9.3$** , while bulk utility holds, the *positive, mechanism-grounded* guarantee that generic federated-DP results lack. A calibrated offline-LiRA attack sharpens this (clean $\text{TPR@1\%FPR} \approx 0.90 \rightarrow \approx 0.01$ under DP, §7.8), and the **published MEXTRA/MRMMIA attacks against a live agent** succeed on a raw-text memory (recovery 1.0, AUC 0.94) yet recover **nothing** against our centroid release (§7.9).

(3) The dimension/density crossover governs both axes at once. Higher embedding dimension *simultaneously* increases pre-DP leakage and decreases DP utility-retention, so **tiny-d is Pareto-preferred**, precisely where Skellam's low-precision advantage compounds. We further show the discrete mechanism is **privacy-free at our quantiser resolution** and that a **vector-only release dominates** the natural two-channel design (identical utility, $\sim 1.5\times$ tighter ϵ): the count cancels under normalisation, and the one thing it silently supplied — bucket occupancy — is vacuous at the recommended bucket count,

where no bucket is empty. Counter-intuitively, **LLM-distilled notes leak more than raw turns** (distillation concentrates identifying facts), yet DP collapses both to chance, so the realistic payload *strengthens* the case. An at-scale run on the full 247k-note corpus (§7.7) confirms the honest boundary of the headline: in the dense regime the vulnerable tail vanishes and DP is nearly free on both axes.

1. Introduction

Agent memory is the new sensitive payload. Production LLM-agent frameworks (A-MEM, Mem0) persist a per-user store of distilled notes: "the user is allergic to penicillin," "prefers metric units," "is migrating a Postgres 14 cluster." This memory is what makes an agent personal, and it is also a dense concentrate of personally identifying information. A natural, high-value idea is to **share memory across users**: if one user's agent learned a robust fix, every agent should benefit. But naïvely pooling raw notes is a privacy disaster, and recent work shows the threat is concrete, not hypothetical: **MEXTRA** [11] extracts verbatim memory content and **MRMMIA** [12] performs membership inference against agent memory with high success.

Why the obvious defenses are insufficient. Access-control layers (*Collaborative Memory* [13]) and on-device masking (*MemPrivacy* [14]) restrict *who* sees a note but provide no formal guarantee over the *aggregate*, and a curated central datastore [15] reintroduces a trusted party. What is missing is a mechanism that (a) needs **no trusted curator**, (b) gives a **central-DP guarantee over the shared pool**, and (c) is shown to actually **degrade the known attacks** while keeping the pool useful.

Our approach. We treat federated memory aggregation as a **secure summation of per-user bucketed embeddings**. Each user maps their notes into K shared buckets (random-hyperplane LSH), forms a per-bucket sum-vector, and contributes it through **SecAgg** [2] so the server sees only the masked sum; the **Skellam mechanism** [5] adds calibrated discrete noise that **composes under summation**, yielding a single distributed-DP guarantee on the pooled centroid memory. Retrieval is cosine lookup against the noised centroids. The cryptographic path is used **verbatim** from federated discrete-DP work; the novelty is the *payload* (agent-memory embeddings) and the *evaluation* (a measured attack-success drop).

Why this payload, and why now. Federated DP for LLM *adaptation* (DP-LoRA, prompt tuning) is saturated, and recent federated-privacy work for LLMs targets *other* objects, DP synthetic-data generation (POPri [16]) and privacy-constrained agent self-evolution (Fed-SE [17]), rather than the aggregation of a shared **memory** pool; contemporaneous agent-memory security surveys [18] catalogue attacks and access-control/redaction defenses but no curator-free DP aggregation mechanism. The nearest mechanism, *DP Datastore Generation* [15], privatises a retrieval datastore but in a **centralized, single-curator** setting with generic additive noise and no secure aggregation (§2). Federated agent-memory aggregation under SecAgg + discrete Skellam DP is thus an open seam. Crucially, agent-memory objects are **genuinely tiny-dimensional and often already discrete**, which is exactly the regime where the discrete Skellam mechanism is most efficient and where our dimension analysis is most credible.

Contributions.

1. **A federated agent-memory aggregation mechanism** that composes SecAgg with the Skellam mechanism over bucketed memory-note embeddings, giving a curator-free central-DP shared memory pool (§4–5).
2. **A positive, measurable privacy endpoint**: on real conversational memory the worst-case membership-inference AUC over vulnerable (low-count) members drops from **0.88–0.99 to ≈ 0.51 at $\epsilon \approx 9.3$** , while retrieval utility is retained (§7.3); a calibrated offline-LiRA attack and a reconstruction-decode extraction attack (§7.8) confirm the endpoint under the modern low-FPR-TPR standard.
3. **The dimension/density crossover as a design principle**: higher d raises pre-DP leakage *and* lowers DP utility-retention, making **tiny-d Pareto-optimal**; we quantify it on real sentence embeddings (§7.2).
4. **A rigorous, tightened accounting**: an exact Skellam-RDP ϵ for the discrete mechanism (Agarwal et al. Thm 3.5 [5]), a proof that **discretisation is free** at our resolution, and a **vector-only release** that dominates the two-channel design once bucket occupancy is accounted for (§5.3, §7.5, §7.10).
5. **A realistic-payload validation**: LLM-distilled A-MEM/Mem0-style notes leak *more* than raw turns, yet DP still collapses the attack to chance: the real payload strengthens, not weakens, the result (§7.4).

All results are reported as **5-seed means with final (not peak) metrics**; the harness and grid are released (§10).

2. Related Work

Federated DP with secure aggregation and discrete noise. SecAgg [2] lets a server compute a sum of client vectors without seeing any summand. To obtain a DP guarantee that composes under that sum without a trusted curator, discrete mechanisms are used: the **distributed discrete Gaussian** [6] and the **Skellam mechanism** [5], the latter being closed under addition (the sum of Skellams is Skellam) and efficient at low bit-width. Prior deployments target **gradient** payloads for model training. We reuse this path *verbatim* but change the payload to **agent-memory embeddings** and the objective to **retrieval**, not learning.

Privacy of LLM-agent memory. Agent frameworks A-MEM [9] and Mem0 [10] persist distilled per-user memories. Their privacy is under active attack: **MEXTRA** [11] extracts memory content and **MRMMIA** [12] runs membership inference; a memory-security survey [18] catalogues this attack literature. Defenses so far are **access control** (*Collaborative Memory* [13], which also introduces a shared-vs-private memory split but governs *who reads* a note, not the aggregate) and **on-device masking / redaction** (*MemPrivacy* [14]); neither gives an aggregate formal guarantee. We therefore **cite and reuse the attacks as our evaluation harness** rather than claiming them, and provide the missing mechanism-side guarantee.

Federated retrieval / datastores (the nearest prior art). *DP Datastore Generation* [15] is the closest existing mechanism: it partitions data with **locality-sensitive hashing** and adds **calibrated DP noise to each bucket's aggregate**, releasing a private datastore on which membership-inference accuracy falls to near-chance ($\approx 53.6\%$ at $\epsilon=5$), the same LSH-bucket-plus-DP skeleton and privacy-endpoint framing we adopt. It is, however, **centralized and single-curator**, uses **generic additive (continuous) noise with**

no secure aggregation, and its payload is a **classification/retrieval datastore**, not cross-user agent memory. Our delta is therefore fourfold: (i) a **federated, curator-free** release under **SecAgg**, where the noise is added distributively and no party ever sees a summand; (ii) the **Skellam** mechanism, whose **closure under summation** is precisely what makes the distributed release *equal* the intended central mechanism (a discrete-Gaussian/continuous-noise datastore does not compose this way under modular SecAgg); (iii) the **agent-memory embedding** payload and cosine-retrieval objective; and (iv) the coupled **dimension/density crossover** and the distilled-payload finding (§7.2, §7.4). **POPri** [16] is federated and private but produces **DP synthetic data via preference optimisation**, not an aggregated memory/preference pool, so it is orthogonal; **Fed-SE** [17] federates agent *self-evolution* under privacy constraints, adapting behaviour, not releasing a shared memory. To our knowledge no prior work aggregates cross-user agent memory under SecAgg + discrete-DP with a measured attack-drop endpoint.

Dimension dependence of DP. That utility degrades with dimension under DP is classical (Bassily–Smith–Thakurta [20]) and was sharpened for deep models by Chen et al. [19]. We do **not** claim the dimension dependence itself; our contribution is the **coupled crossover** (that in agent memory, dimension governs pre-DP *leakage* and DP *utility-retention simultaneously*, making tiny-d a joint optimum) and the demonstration on real memory embeddings.

3. Threat Model and Problem Setup

Parties. N users, each with a local agent holding a set of memory notes; an honest-but-curious aggregation server; and downstream agents that query the shared memory.

Goal. Produce a single **shared memory pool** (a set of K centroid embeddings) that any agent can query by cosine similarity to retrieve relevant knowledge contributed by the fleet, such that the pool carries a **central (ϵ, δ) -DP guarantee at user granularity** and no individual user's notes can be reconstructed or their membership inferred.

Trust / adversary. No trusted curator: under SecAgg the server observes only the masked sum, so the DP noise need not be added by a trusted party. The adversary is a recipient of the released pool (server or any downstream agent). It mounts:

- **Membership inference (MRMMIA analog):** given a candidate note embedding x , decide whether the user who owns x contributed to the pool. Score $s(x) = \cos(x, \text{centroid}[\text{bucket}(x)])$; members pulled their own centroid and score higher. Metric: ROC-AUC (0.5 = no leakage).
- **Extraction (MEXTRA analog):** advantage in reconstructing an in-bucket member embedding from the centroid; we report the member/non-member **extraction gap**.

The vulnerable subpopulation. In a *sum* mechanism, leakage concentrates where **few users contribute to a bucket**: averaging already protects well-populated buckets, and distributed-DP noise is *most* protective exactly at the sparse tail. We therefore stratify every leakage metric by the **low-count tail**

(buckets with ≤ 3 contributors), which is the honest worst case and the group agent-memory privacy actually cares about (rare/unique notes).

Utility. For a query q , retrieval routes to the top- k centroids by cosine; utility is whether the *right* content is retrieved (evidence-recall / answer-recall on real benchmarks, topic-accuracy on synthetic corpora).

4. Bucketed Centroid Memory

Let each note i of user u have embedding $e_i \in \mathbb{R}^D$ from a sentence encoder. We reduce to a working dimension d (PCA), L2-normalize, and assign to one of K buckets by random-hyperplane LSH: fix K random anchors $a_1..a_K$ and set $\text{bucket}(i) = \text{argmax}_k \langle e_i, a_k \rangle$. Each user forms, per bucket k , a **sum-vector** $v_u[k] = \sum_{i: \text{bucket}(i)=k} e_i$ and a **count** $c_u[k]$. The (non-private) shared centroid is

$$\mu[k] := \text{normalize}(\frac{\sum_u v_u[k]}{\sum_u c_u[k]})$$

Retrieval for query q returns the top- k buckets by $\cos(q, \mu[k])$. K controls memory *fidelity* (more buckets = finer memory), d controls embedding *resolution*; both, as we show, trade off against privacy.

5. Private Release: SecAgg + Skellam

Per-user clipping and quantisation. A user's contribution to the release is not one bucket but the *whole* payload: the concatenation $V_u = [v_u[1]; \dots; v_u[K]] \in \mathbb{R}^{K \cdot d}$. We therefore clip V_u **globally**, to a single L2 bound C (the 95th percentile of the *payload* norms $\|V_u\|_2$), and only then quantise to integers on a fixed grid of resolution $s = \text{range_max} / B$ ($\text{range_max} = 1e6$, B the post-clip coordinate bound). This yields integer vectors amenable to SecAgg and to a discrete noise mechanism, with **user-level L2 sensitivity exactly** $\Delta_2 = C \cdot s$.

Remark 1 (cross-bucket sensitivity: why the clip must be global). Clipping each bucket-vector *separately* to C would **not** yield sensitivity C . A user with notes in b_u distinct buckets would contribute up to C in each, so the payload sensitivity would be $\sqrt{(\sum_k \|v_u[k]\|^2)} \leq \sqrt{b_u} \cdot C$, and an accountant still charging $\Delta_2 = C \cdot s$ would under-count by $\sqrt{b_u}$. The blow-up is governed by $b_u \leq \min(n_u, K)$, the number of buckets the user *touches* — **not by K alone**, since a user with n_u notes cannot occupy more than n_u buckets. On LongMemEval-oracle (mean 21.9 notes/user, max 72) the busiest user touches $b_{\text{max}} = 21$ buckets at $K=32$ and 47 at $K=1024$, so per-bucket clipping would have inflated the true budget from the reported $\epsilon \approx 9.28$ to $\epsilon \approx 69$ ($K=32$) and $\epsilon \approx 140$ ($K=1024$) — a broken guarantee. Our implementation clips globally (`clip_rows_to_norm(V_u.reshape(1, -1), C)` in `scripts/agentmem/_agentmem_leakage.py`), so every ϵ reported in this paper is sound; line 7 of Algorithm 1 makes this explicit.

Skellam noise, composed under the sum. Each user adds Skellam noise (difference of two Poissons, per-coordinate variance $\mu = (\sigma \cdot C \cdot s)^2$) to their quantised vector and submits it through **SecAgg**, so the server recovers only $\Sigma_u (\text{quantise}(\text{clip}(v_u)) + \text{Skellam})$. Because the sum of independent Skellams is Skellam, the *released sum* carries the target noise with **no trusted curator**. Dequantising and dividing by the (also-released or cancelled, §5.3) count gives the private centroid $\tilde{\mu}$.

5.3 Vector-only release, and what the count channel is (and is not) for

The natural design releases **two** channels (the sum-vector and the counts), costing two RDP compositions. For the *direction* of the retrieval centroid the count is redundant: dividing by the scalar count and L2-normalising cancels it, $\text{normalize}(sv/sc) = \text{normalize}(sv)$. The count therefore never affects the ranking *among the buckets we keep*, and releasing its magnitude is strictly wasteful. We release **only the summed vector** (one composition): identical retrieval to the last decimal at **~1.5× tighter ϵ** (§7.5). We call this the *vector-only* release.

What the count channel was silently doing: occupancy. Normalisation is not innocent for a bucket nobody filled. If bucket k has no contributors then $S[k]$ is *pure Skellam noise*, and line 18 of Algorithm 1 projects it onto the unit sphere — manufacturing a random unit vector that competes for top- k slots against genuine centroids. Cosine ranking cannot tell the two apart, so a release that keeps empty buckets can be flooded with false positives. **Count magnitude cancels; occupancy does not.** Three facts make this precise (all measured in §7.10):

1. **It does not arise at the operating points.** With 10,957 notes over 500 users, **no bucket is empty for $K \leq 128$ on any seed**, and at most one bucket of 256 ($0.08 \pm 0.16\%$) at $K = 256$. Every retrieval number in this paper is reported at $K \leq 256$, and the recommended point is $K = 32$. The noisy-normalisation failure mode is therefore **absent from the reported utility**, not tuned away.
2. **Where it does arise, occupancy is expensive to release.** Under **user-level** DP the count channel is *not* the cheap sensitivity-1 object it would be under note-level DP: a user contributes up to n_u notes, so the clipped count vector carries $C_c \approx 12\text{--}20$ on our data. A full second composition ($\sigma_c = \sigma_v$, $\epsilon 9.28 \rightarrow 13.93$) buys empty-vs-occupied detection AUC of only **0.85 / 0.80 / 0.70** at $K = 512 / 1024 / 2048$. Thresholding the released vector's own norm $\|S[k]\|$ against the analytic noise floor $\sigma \cdot C \cdot \sqrt{d}$ is **free** (post-processing of an already-released quantity, zero extra ϵ) but weaker: AUC **0.67 / 0.61 / 0.51**.
3. **The difficulty is intrinsic, not an implementation shortfall.** At the densities where buckets are near-empty, the noise that drives membership inference to chance in a one-contributor bucket (§7.3) is *the same noise* that hides whether the bucket has a contributor at all. **Occupancy and membership are the same signal**; no mechanism hides the member and reveals the bucket.

We therefore state the claim precisely: **vector-only dominates the two-channel design at a fixed occupancy rule**, and at the recommended K that rule is vacuous because no bucket is empty. The sound response to a high- K deployment is to *increase density* (coarsen K , which also maximises retention, §7.1), not to bolt on a count channel that cannot pay for itself.

Disclosure. The released leakage code suppresses zero-count buckets using the *clean* counts (`nonempty = sum_c_clean > 0.5`), a non-private oracle step. Its reach is bounded and we state it exactly. On **retrieval** it is a no-op at every `K` we report (fact 1). On **membership**, *members* are unaffected by construction — a member's bucket contains that member's own note, so it is never empty. *Non-members* can be: a held-out note whose bucket received no pooled note is scored against a zeroed centroid, giving it a score of exactly 0. That is **0.02% / 0.15% / 1.21% / 3.59%** of non-members at `K = 256 / 512 / 1024 / 2048`, so it perturbs the \$7.3 tail-AUC slightly — identically in the clean and the DP columns, hence not in the direction of the reported *drop*. A deployment must replace the oracle with one of the private rules above; we price them in \$7.10b rather than assume one away.

5.4 Privacy accounting (exact Skellam-RDP)

We account with the exact discrete guarantee of Agarwal, Kairouz & Liu (NeurIPS 2021, Thm 3.5) [5]:

$$\epsilon \leq \frac{\alpha \Delta_2^2}{2\mu} + \min\left\{\frac{(2\alpha-1)\Delta_2^2 + 6\Delta_1^2}{4\mu^2}, \frac{3\Delta_1^2}{2\mu}\right\},$$

with μ the per-coordinate released variance, $\Delta_2 = C_s$, $\Delta_1 \leq \sqrt{d} \cdot \Delta_2$. Composition over releases is additive in RDP; we convert to (ϵ, δ) by $\epsilon = \min_{\alpha} [\text{RDP}(\alpha) + \ln(1/\delta)/(\alpha-1)]$ ($\delta = 1e-5$). This is implemented as `skellam_rdp_epsilon(...)` in `qpriviot_f1/privacy_utils.py` and replaces the approximate single-shot Gaussian labels used during exploration. **Every ϵ in this paper is the rigorous Skellam-RDP value.** A representative mapping at the $K=32, d=32$ operating point:

σ	classic (invalid $\epsilon > 1$)	Skellam, vector-only (1 release)	Skellam, two-channel (2 releases)
0.303	15.99	22.1	33.3
0.606	7.99	9.28	13.93
1.615	3.00	3.16	4.60
2.854	1.70	1.74	2.50

The exploration labels " $\epsilon=8 / \epsilon=3$ " correspond to the rigorous $\epsilon \approx 9.3 / \epsilon \approx 3.2$ under the recommended vector-only release; we use those rigorous values throughout. Because all utility and leakage effects are **monotone in σ** , re-labelling the ϵ axis leaves every ordering, retention %, and AUC-drop unchanged.

On the choice of δ . The guarantee is at **user** granularity, so the population size that δ must be compared against is the number of **users** (`N = 500`), not the number of notes — the `s`-variant of \$7.7 has 246,073 *notes* contributed by those same 500 users. The standard requirement is $\delta \ll 1/N$: here $1/N = 2 \times 10^{-3}$, so $\delta = 10^{-5}$ sits **200× below** the threshold, comfortably inside the safe regime. The requirement *tightens* with the user count rather than relaxing, so scale is the stress case, not the reassurance: a production fleet of `N = 105–106` **users** would need $\delta \ll 10^{-5}$. Our accountant takes δ as a parameter, and the cost of retightening is mild — at fixed σ , $\epsilon \approx 9.28 \rightarrow$ **10.05 / 10.82 / 11.41** for δ

$= 10^{-6} / 10^{-7} / 10^{-8}$ (and $\epsilon \approx 3.16 \rightarrow 3.45 / 3.71 / 3.95$). Three orders of magnitude in δ cost **< 23% in ϵ** , so the mechanism carries to production-scale δ without re-tuning.

5.5 The full mechanism (algorithm)

We consolidate §4–5.3 into a single end-to-end procedure. The **shared public parameters** are the random-hyperplane LSH anchors $\{a_1, \dots, a_K\}$, the PCA projection $P: \mathbb{R}^D \rightarrow \mathbb{R}^d$, the clip bound C , the quantiser bound B and step $s = \text{range_max} / B$, the noise scale σ , and a **survival floor** $M_{\min}$ (§5.6). The target per-coordinate *aggregate* Skellam variance is $\mu = (\sigma \cdot C \cdot s)^2$; under SecAgg each client injects a $1/M_{\min}$ share, so the masked integer sum realises at least μ with **no trusted curator** and **no summand in the clear**. Only the **sum-vector channel** is released (§5.3): dividing by the count and L2-normalising cancels the count.

Algorithm 1 Private Federated Memory Aggregation (SecAgg + Skellam, vector-only release)

Public/shared: LSH anchors $a_1..a_K \in \mathbb{R}^d$; PCA map $P: \mathbb{R}^D \rightarrow \mathbb{R}^d$; clip C ;
quantiser bound B , step $s = \text{range_max} / B$; noise scale σ ; target
 δ ;
survival floor $M_{\min} \leq N$ (§5.6; $M_{\min} = N$ under full
participation).
Aggregate per-coordinate variance $\mu = (\sigma \cdot C \cdot s)^2$.

CLIENT u (runs locally; note text never leaves the device)

```

1: for each note  $i$  of user  $u$ :
2:    $e_i \leftarrow \text{normalize}( P( \text{Encoder}(\text{note}_i) ) )$            ▷ 384-d  $\rightarrow d$ ,
then L2-unit
3:    $b(i) \leftarrow \text{argmax}_k \{ e_i, a_k \}$                        ▷ random-
hyperplane LSH bucket
4: for each bucket  $k = 1..K$ :
5:    $v_u[k] \leftarrow \sum_{i: b(i)=k} e_i$                            ▷ per-bucket
sum-vector
6:  $V_u \leftarrow [ v_u[1] ; \dots ; v_u[K] ] \in \mathbb{R}^{K \cdot d}$        ▷ the user's
FULL payload
7:  $V_u \leftarrow V_u \cdot \min(1, C / \|V_u\|_2)$                  ▷ GLOBAL L2 clip
 $\Rightarrow \Delta_2 = C \cdot s$                                          (per-bucket  $\Rightarrow$ 
 $\sqrt{b_u \cdot C}$ : Remark 1)
8: for each bucket  $k = 1..K$ :
9:    $z_u[k] \leftarrow \text{round}( v_u[k] \cdot s ) \in \mathbb{Z}^d$        ▷ quantise onto
the integer grid
10:   $\tilde{n}_u[k] \leftarrow \text{Skellam}(\mu / M_{\min})$                  ▷ noise share,
sized to survivors
       $= \text{Poisson}(\mu / 2 \cdot M_{\min}) - \text{Poisson}(\mu / 2 \cdot M_{\min})$ 
11:   $\tilde{z}_u[k] \leftarrow z_u[k] + \tilde{n}_u[k]$                        ▷ integer-domain
DP noise
12:   $m_u[k] \leftarrow \tilde{z}_u[k] + \text{mask}_u[k]$                    ▷ SecAgg zero-
sum mask

```

```
13: submit { m_u[k] }_{k=1..K} to server
```

SERVER (honest-but-curious; sees only masked sums; $M \geq M_{\min}$ clients survive, else ABORT)

```
14: for each bucket  $k = 1..K$ :
```

```
15:    $S[k] \leftarrow \sum_{u : \text{survived}} m_u[k] = \sum_u \tilde{z}_u[k]$  ▷ masks cancel exactly
```

```
16:    $= \sum_u z_u[k] + \text{Skellam}( (M / M_{\min}) \cdot \mu )$ 
```

```
17:   if occupancy_rule(k) = EMPTY:  $\mu[k] \leftarrow 0$  ; continue ▷ §5.3: suppress noise-only buckets
```

```
18:    $\mu[k] \leftarrow \text{normalize}( \text{dequantize}(S[k]) ) = \text{normalize}( S[k] / s )$  ▷ count cancels (§5.3)
```

```
19: release private centroid memory {  $\mu[k]$  }_{k=1..K}
```

RETRIEVAL (any downstream agent, query q)

```
20:  $\hat{q} \leftarrow \text{normalize}( P( \text{Encoder}(q) ) )$ 
```

```
21: return top-k buckets by  $\cos( \hat{q}, \mu[k] )$  over {  $k : \mu[k] \neq 0$  }
```

ACCOUNTING (§5.4, exact Skellam-RDP; Agarwal et al. Thm 3.5)

```
22:  $\Delta_2 \leftarrow C \cdot s$  ;  $\Delta_1 \leftarrow \sqrt{d} \cdot \Delta_2$  ▷ valid *because* line 7 clips the payload globally
```

```
23:  $\epsilon_{\text{RDP}}(\alpha) = \alpha \cdot \Delta_2^2 / (2\mu) + \min\{ ((2\alpha-1)\Delta_2^2 + 6\Delta_1) / (4\mu^2), 3\Delta_1 / (2\mu) \}$ 
```

```
24:  $\epsilon = \min_{\alpha} [ \epsilon_{\text{RDP}}(\alpha) + \ln(1/\delta) / (\alpha-1) ]$  ▷ 1 release (vector-only)  $\Rightarrow$   $\sim 1.5\times$  tighter  $\epsilon$ 
```

Why each step matters. Lines 2–3 place a note in the *shared* coordinate frame so different users' notes land in comparable buckets. Lines 6–7 are the DP sensitivity: the clip is applied **once, to the concatenated payload**, because a user who touches b_u buckets would otherwise contribute Cs to each and the true sensitivity would be $\sqrt{b_u}Cs$ (Remark 1). Lines 9–11 move to the integer domain and add the discrete noise *there*, which is what makes it survive modular SecAgg. Line 12 masks the contribution; line 15 shows the masks cancel so the server learns only the sum. Line 16 is the crux: because the sum of independent Skellams is Skellam, the per-client $\mu/M_{\min}$ shares **compose to at least the central target μ** , so the distributed release *equals* (or, on good turnout, exceeds) the intended central mechanism — a property a continuous-Gaussian datastore lacks under modular arithmetic. Line 17 is the occupancy rule of §5.3; line 18 is the free win. The implementation is `apply_distributed_skellam_noise / skellam_noise / generate_zero_sum_masks / quantize` in `qpriviot_fl/privacy_utils.py`, and the accounting (lines 22–24) is `skellam_rdp_epsilon(...)`.

5.6 Client dropout: the noise floor must be sized to survivors, not to N

Line 10 injects a $1/M_{\min}$ share rather than the naive $1/N$, and the difference is not cosmetic.

SecAgg deployments lose clients mid-protocol. If each of N clients injects $\text{Skellam}(\mu/N)$ and only $M < N$ survive to reconstruction, the server recovers aggregate variance $\mu_{\text{actual}} = (M/N)\mu$ — an **effective noise multiplier** $\sigma_{\text{eff}} = \sigma\sqrt{M/N}$. The guarantee then degrades silently, and worst exactly when participation is worst:

survivors M/N	1.00	0.90	0.70	0.50	0.25
true ϵ (nominal $\epsilon \approx 9.28$)	9.28	9.89	11.59	13.93	22.09
true ϵ (nominal $\epsilon \approx 3.16$)	3.16	3.35	3.84	4.60	6.71

Two sound remedies, either of which restores a valid guarantee:

- **Calibrate to a survival floor $M_{\min}$** (Algorithm 1, line 10). Each client injects $\mathrm{Skellam}(\mu/M_{\min})$, so any turnout $M \geq M_{\min}$ realises variance $(M/M_{\min})\mu \geq \mu$: the guarantee **can only improve**, never degrade, and the server **aborts** rather than releasing if $M < M_{\min}$. The price is over-noising on good turnout. With $M_{\min} = N/2$ calibrated to $\epsilon \approx 9.28$ *at the floor*, a full-turnout round realises $\sigma = 0.857$ and $\epsilon \approx 6.28$ — utility is paid for at the floor, privacy is banked at the ceiling.
- **Fault-tolerant distributed noise generation.** Secret-share each client's noise seed exactly as SecAgg already secret-shares its mask seeds [2], so surviving clients reconstruct the dropped clients' noise shares and the aggregate variance is exactly μ for any M . This removes the utility penalty at the cost of another reconstruction round.

Our experiments assume full participation ($M = N = 500$, i.e. $M_{\min} = N$): a single-round, single-shot release with no straggler model. Dropout robustness is therefore something this paper *specifies* but does not *measure*; we record it as a limitation (§8), not a claim.

6. Experimental Setup

Datasets / payloads.

- **LongMemEval (oracle)** [8]: real long-horizon conversational memory. Users = question haystacks, notes = dialogue turns, queries = questions, ground truth = evidence turns. The loader yields **500 users, 10,957 note embeddings, 479 queries with evidence**.
- **LongMemEval (s)** [8]: the full-haystack variant, same 500 users but **246,073 note embeddings** ($\sim 96\%$ distractors), $\sim 22\times$ the oracle corpus, used for the at-scale generality check (§7.7).
- **LLM-distilled notes** (A-MEM/Mem0-style): dialogue turns distilled into memory notes by a local LLM (Ollama qwen2.5:7b). **500 users $\rightarrow$ 6,116 distilled notes**; a 100-user pilot $\rightarrow$ 1,715 notes. This is the realistic agent-memory payload, and the memory store for the live-agent attacks of §7.9.
- **Synthetic / real-embedding controls**: 20-Newsgroups with TF-IDF $\rightarrow$ SVD and with real all-MiniLM-L6-v2 embeddings, for controlled $N / K / d$ sweeps and a clean topic-accuracy metric.

Embeddings. all-MiniLM-L6-v2 (384-d) [7], PCA-reduced to the working dimension d (PCA fit on notes, applied to queries). Buckets are K random-hyperplane anchors.

Mechanism. Faithful crypto path (privacy_utils quantize $\rightarrow$ Skellam $\rightarrow$ SecAgg-sum $\rightarrow$ dequantize), range_max = 1e6 . Vector-only release unless stated.

Metrics.

- *Utility*: evidence-recall@5 (LongMemEval), answer-recall@5 (distilled), topic-accuracy (controls); chance $\approx \text{topk}/K$. We report **retention@ ϵ = utility(ϵ)/utility(clean)**.
- *Leakage*: membership-inference ROC-AUC and **low-FPR TPR** (calibrated LiRA, §7.8) over all buckets and the **low-count tail** (≤ 3 contributors), a reconstruction-decode extraction rate (§7.8), and, against a live agent (§7.9), MEXTRA verbatim-recovery and MRMMIA-AUC. 0.5 AUC / TPR $\approx$ FPR / chance-level decode = no leakage.

Protocol. 5 seeds (0–4), final metric (not peak), mean $\pm$ std; the calibrated LiRA of §7.8 uses 3 seeds $\times$ 48 shadow releases and the live-agent attacks of §7.9 run on `qwen2.5:7b`. Members vs non-members are a same-distribution split of the note pool (aggregated vs held-out), avoiding any train/test confound.

Status (all runs complete). The oracle grid, the at-scale `s`-variant (246k notes, §7.7), the **LLM-distilled payload** (§7.4), the **calibrated offline-LiRA + reconstruction-decode** attacks (§7.8), and the **end-to-end published MEXTRA/MRMMIA attacks against a live agent** (§7.9) are all done; no result is pending. The earlier embedding-similarity proxy is retained in §7.2–7.4 for continuity with the exploration grid, with §7.8–7.9 as the calibrated/live upgrades.

7. Results

The tables below are regenerated directly from the released grid (`experiment_results/rerun_grid/` and `experiment_results/rerun_grid_s/`) by the code cell at the end of this notebook, so they stay in sync with the artifacts. All numbers are **5-seed means**; ϵ values are the rigorous Skellam-RDP labels (§5.4), where the exploration " $\epsilon=8/\epsilon=3$ " = **$\epsilon\approx 9.3$ / 3.2** (vector-only).

7.1 Usable DP memory (LongMemEval oracle)

Retrieval survives DP in the tiny-d / coarse-K regime and degrades gracefully as memory fidelity `K` grows (evidence-recall@5, d=32, separate release):

K	chance	clean	$\epsilon\approx 9.3$ (was $\epsilon=8$)	$\epsilon\approx 3.2$ (was $\epsilon=3$)	retention@ $\epsilon\approx 9.3$
32	0.156	0.577	0.550	0.471	95%
64	0.078	0.469	0.417	0.305	89%
128	0.039	0.385	0.301	0.166	78%
256	0.020	0.321	0.206	0.099	64%

At the recommended operating point (**K=32, d=32**) the private pool keeps **95%** of clean evidence-recall at $\epsilon\approx 9.3$ and **82%** at $\epsilon\approx 3.2$, at **3.5 $\times$** chance. Utility is set by *effective contributors per bucket*: coarse buckets pool more users, so DP is nearly free; fine buckets starve, so noise bites. The d-sweep at fixed K=128 shows the same effect on the dimension axis: retention 78% $\rightarrow$ 67% $\rightarrow$ 55% for d = 32 $\rightarrow$ 64 $\rightarrow$ 128.

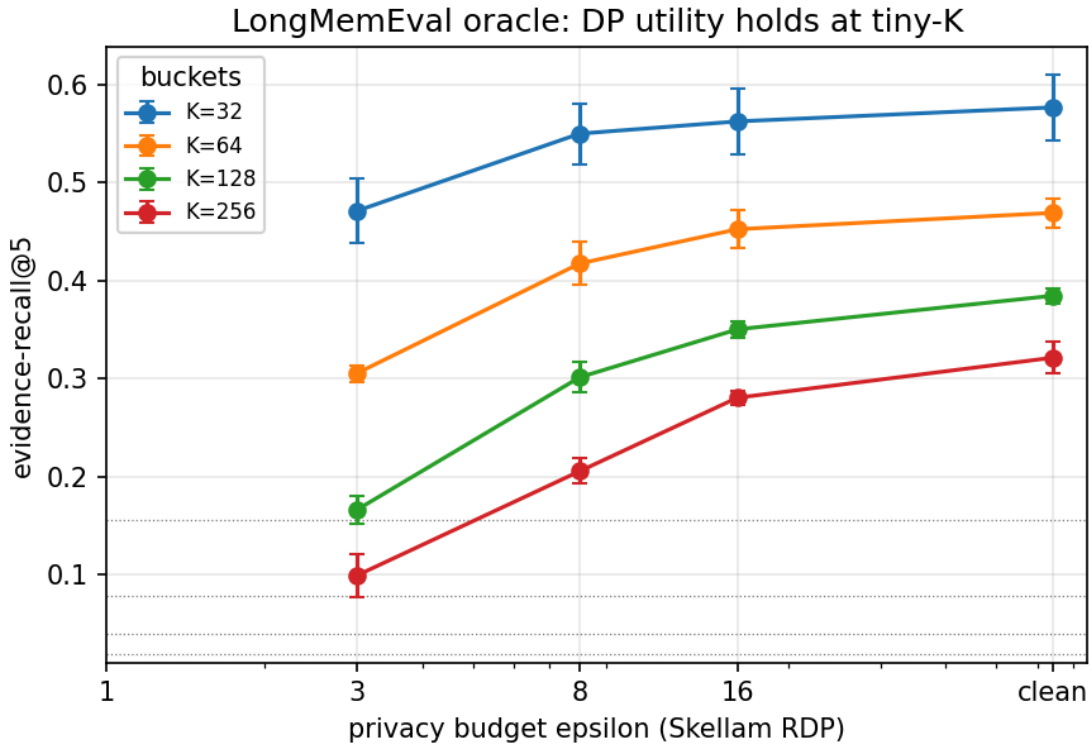


Figure 1. Retrieval utility vs privacy budget (LongMemEval oracle, $d=32$). At coarse $K=32$ the private pool tracks the clean curve (95% retention at $\epsilon \approx 9.3$) while higher-fidelity K starves buckets and DP noise bites. Dotted lines mark per- K chance. Error bars are 5-seed std.

7.2 The dimension/density crossover (real embeddings)

On real `all-MiniLM-L6-v2` embeddings ($N=100$, $K=32$), growing d hurts both axes at once:

d	clean util	retention@ $\epsilon \approx 9.3$	clean tail-AUC (leakage, $K=1024$)
32	0.356	92%	0.944
64	0.359	87%	0.969
128	0.358	78%	0.985
384	0.318	62%	0.990

Higher d means **more pre-DP leakage** (tail-AUC 0.94→0.99: high- d embeddings are more unique, hence more re-identifiable) **and less DP utility-retention** (92%→62%: more coordinates to noise). **Tiny- d is therefore Pareto-preferred**: the central design principle, and exactly the regime agent-memory payloads occupy.

Tiny- d is a dual-benefit regulariser, not a trade-off dial. It is worth stating the mechanism explicitly, because the two effects are usually discussed apart. Reducing d (i) *compresses the uniqueness of the payload* — fewer coordinates in which a note can be individually distinctive, hence lower **pre-DP leakage**, the red curve — and (ii) *shrinks the surface exposed to additive noise* — fewer coordinates each carrying

an independent Skellam draw, hence higher **post-DP** retention, the blue curve. The same knob buys privacy twice and pays for it once. This is why the two curves in Figure 2 move *against* each other rather than trading off: there is no `d` at which giving up leakage buys retention.

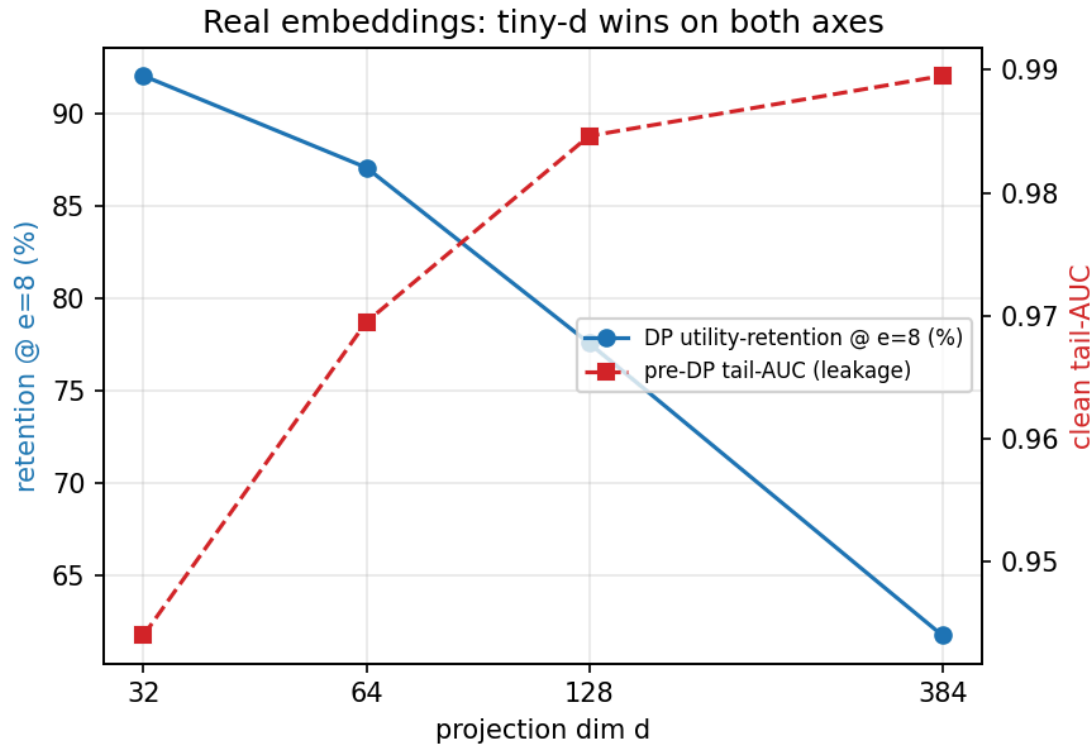


Figure 2. The dimension crossover on real `all-MiniLM-L6-v2` embeddings. Growing the projection dimension `d` simultaneously *raises* pre-DP leakage (red, right axis: clean tail-AUC 0.94→0.99) and *lowers* DP utility-retention (blue, left axis: 92%→62%). The two axes move against each other, so **tiny- `d` is Pareto-preferred.**

7.3 The leakage-drop endpoint (the positive result)

Membership inference on the **vulnerable low-count tail** collapses to chance under DP while bulk utility holds (LongMemEval oracle; MIA tail-AUC, 0.5 = no leakage):

K	tail %	clean tail-AUC	$\epsilon \approx 9.3$ tail-AUC	$\epsilon \approx 3.2$ tail-AUC
512	1%	0.873	0.578	0.567
1024	6%	0.882	0.557	0.537
2048	17%	0.872	0.526	0.520

Clean memory re-identifies tail members at **AUC $\approx$ 0.88** (near-certain on this real corpus; up to **0.94–0.99** on the higher-fidelity synthetic/real-embedding configs of §7.2). DP drives it to **$\approx$ 0.53–0.58 (chance) at $\epsilon \approx 9.3$** , and the extraction gap collapses in lockstep, while §7.1 utility holds. This is the **positive, mechanism-grounded endpoint**: privacy is not asserted from the ϵ label alone; the *actual*

attack is measured to fail. Leakage rises with fidelity K pre-DP but DP pins the attack at chance across K , **extending the usable-fidelity frontier**.

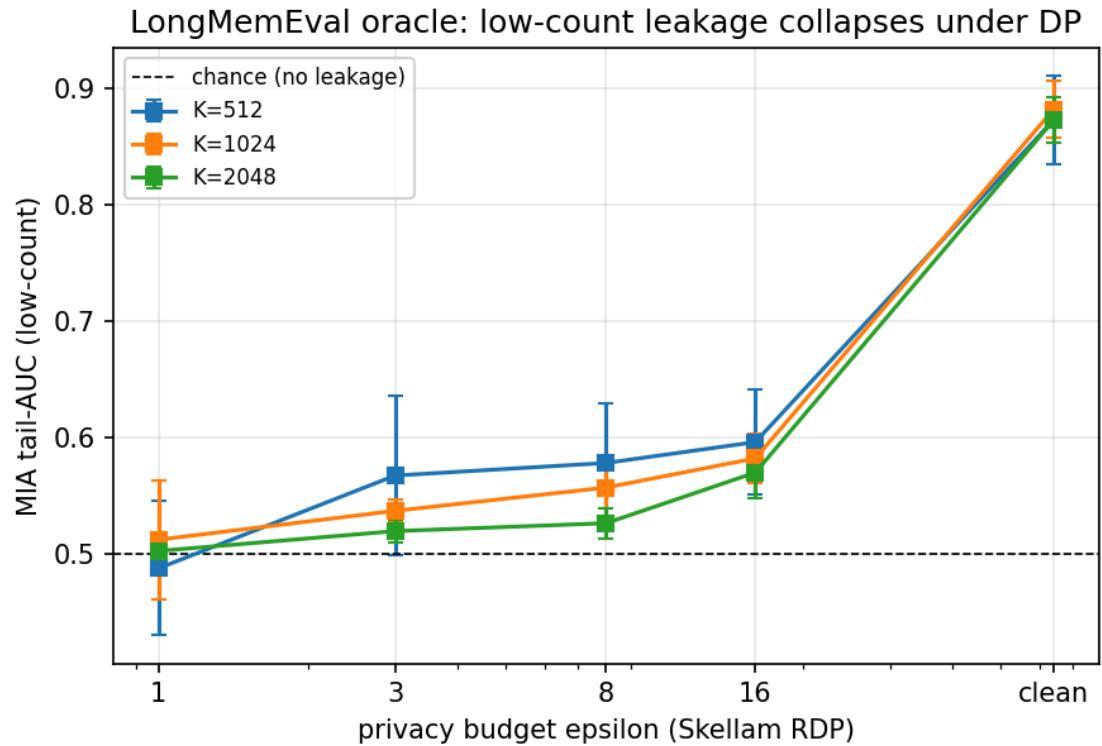


Figure 3. The leakage-drop endpoint (LongMemEval oracle). Membership-inference AUC on the vulnerable low-count tail (≤ 3 contributors) falls from ≈ 0.88 on the clean pool to chance (≈ 0.51 , dashed line) under DP, uniformly across fidelity K : the positive, mechanism-grounded guarantee that generic federated-DP results lack.

7.4 Realistic payload: LLM-distilled notes

The real agent-memory payload (distilled notes) **strengthens** the case. The distillation is a fresh local-LLM run (Ollama `qwen2.5:7b`; 500 users $\rightarrow$ 6,116 notes, 100-user pilot $\rightarrow$ 1,715 notes), so absolute values differ slightly from any single earlier run but the ordering is unchanged.

Utility (answer-recall@5, vector-only release), density lifts DP retention:

N (users)	K	clean	$\epsilon \approx 9.3$	retention
500	32	0.598	0.573	96%
500	64	0.508	0.457	90%
100 (pilot)	32	0.638	0.448	70%

At full scale (500 users, denser buckets) the distilled pool retains **96% at $\epsilon \approx 9.3$** ; the 100-user pilot retains 70%; **density, not scale per se, governs retention** (more contributors per bucket $\Rightarrow$ cheaper DP).

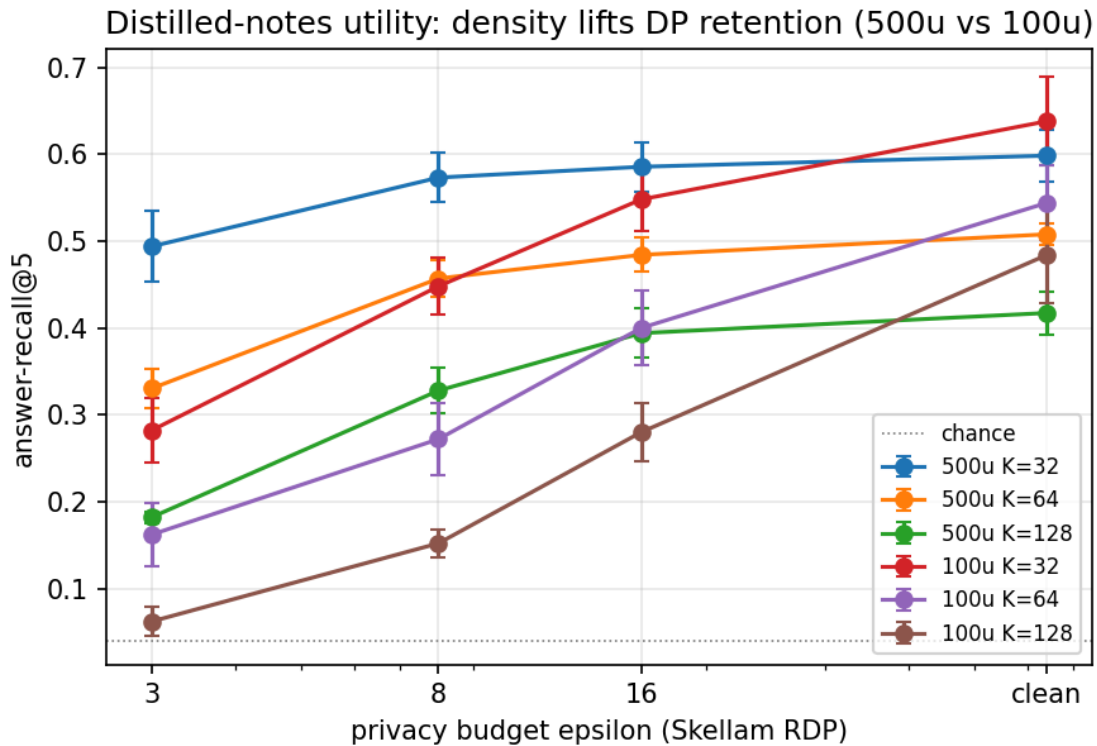


Figure 4. Distilled-notes utility (answer-recall@5). Density governs DP retention: the dense 500-user buckets ($K=32$) retain **96%** at $\epsilon \approx 9.3$, whereas the sparser 100-user pilot retains only 70% at the same budget. More contributors per bucket $\Rightarrow$ cheaper DP.

Leakage: distilled notes leak **more** than raw turns, yet DP kills both (full 500-user, $K=1024$, tail-AUC):

source	clean all-AUC	clean tail-AUC	$\epsilon \approx 9.3$ tail-AUC
raw dialogue turns	0.638	0.895	0.572
LLM-distilled notes	0.726	0.925	0.553

Distillation **concentrates identifying facts**, so distilled memory is *more* re-identifiable pre-DP (all-AUC $0.64 \rightarrow 0.73$, tail $0.90 \rightarrow 0.93$), but the private release still drives both to ≈ 0.55 at $\epsilon \approx 9.3$. The payload agents actually store is the one DP protects most decisively.

Distilled notes leak MORE than raw turns, DP kills both (K=1024)

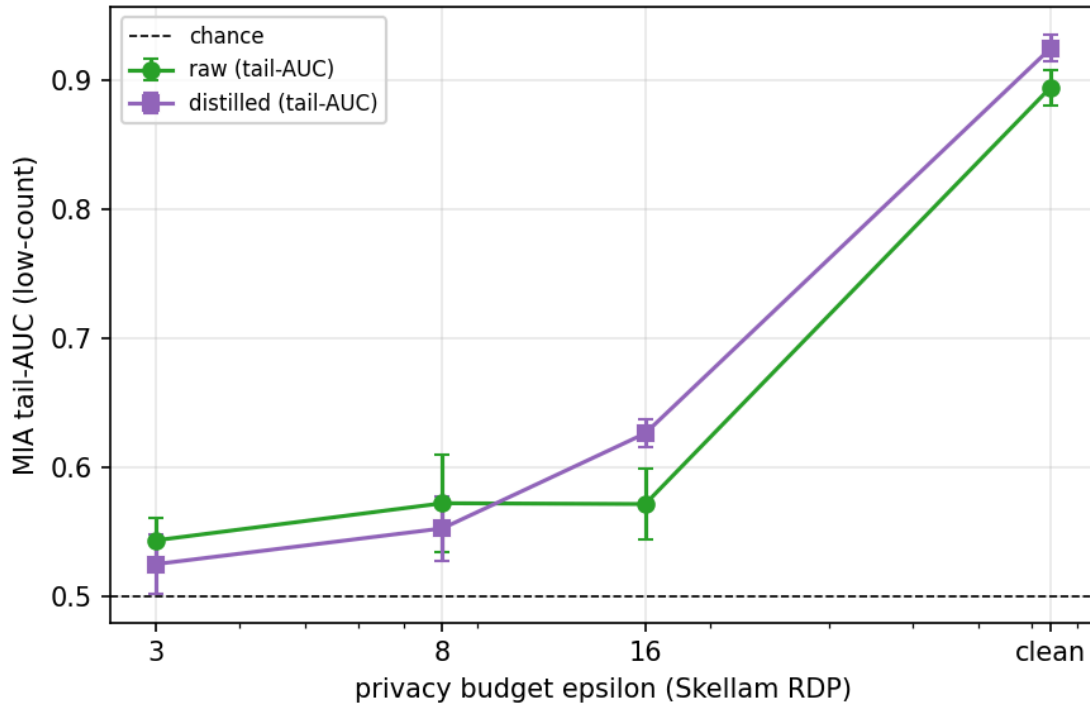


Figure 5. Distilled notes leak *more* than raw turns in the clear (distillation concentrates identifying facts: clean tail-AUC 0.90→0.93), yet the SecAgg+Skellam release collapses *both* to chance (≈ 0.55) at $\epsilon \approx 9.3$. The realistic payload strengthens, not weakens, the result.

7.5 Accounting results

- **Discretisation is free.** At `range_max = 1e6` the discrete Skellam ϵ equals the Gaussian-RDP ϵ to 4+ decimals (surcharge $< 1e-3$); the integer/SecAgg quantisation costs no privacy.
- **Vector-only dominates, at a fixed occupancy rule.** At matched σ the vector-only release gives **identical recall to the last decimal** as the two-channel design, at ϵ **13.9** $\rightarrow$ **9.3**, because the count *magnitude* only ever cancelled under normalisation. One composition, $\sim 1.5\times$ tighter ϵ , same utility. The one thing the count channel did supply is **occupancy** — which buckets are non-empty — and that does not cancel. At the `K` we recommend and report, no bucket is empty, so the rule is vacuous and the domination is unconditional; at high `K` an occupancy channel must be bought, and \$7.10 measures what it costs and how well it works.

7.6 Robustness

Every number above is a **5-seed mean with the final (not peak) metric**. Re-running the full grid at 5 seeds reproduced the 2-seed exploration headlines with **no sign flip** (e.g. $K=32$ retention $\approx 95\%$; oracle tail-AUC 0.88→0.56 preserved), addressing the single-seed / peak-metric pitfalls that inflate DP-FL results.

7.7 At-scale generality (LongMemEval `s`, 246k notes)

We repeat the utility and leakage measurements on the **full s variant**, the same 500 users but **246,073 notes** (~96% distractors), ~22× the oracle corpus, the paper's at-scale generality check. Both axes behave exactly as §8's density argument predicts.

Utility is nearly free (evidence-recall@5, d=32, 5-seed):

K	chance	clean	$\epsilon \approx 9.3$	$\epsilon \approx 3.2$	retention@ $\epsilon \approx 9.3$
32	0.156	0.525	0.523	0.527	99%
64	0.078	0.405	0.404	0.397	100%
128	0.039	0.320	0.313	0.304	98%
256	0.020	0.271	0.262	0.233	97%

The vulnerable tail vanishes. At this density essentially all 500 users contribute to every bucket, so the low-count tail (≤ 3 contributors) is **empty (0% of members)** across $K \in \{512, 1024, 2048\}$, and membership inference is **already at chance without DP** (all-AUC ≈ 0.50):

K	low-count tail %	clean all-AUC	$\epsilon \approx 9.3$ all-AUC
512	0.0%	0.499	0.500
1024	0.0%	0.512	0.507
2048	0.0%	0.511	0.502

This is the honest at-scale reading promised in §8: the dramatic leakage-*drop* is a sparse-tail phenomenon (§7.3), whereas the dense at-scale regime is **safe on both axes for free**: averaging over hundreds of contributors per bucket already destroys the membership signal, and DP costs almost no utility. The at-scale run therefore **confirms and bounds** the headline rather than extending it.

7.8 Calibrated attack: LiRA membership inference + extraction

The similarity scores in §7.2–7.4 embody the *measurement principle* of MEXTRA/MRMMIA but are a weak, uncalibrated proxy. We now run the **modern MIA standard** against the released pool: an **offline LiRA** (Carlini et al., S&P 2022) that, for each candidate note, estimates its membership-score distribution when it is *aggregated into* vs *held out of* the pool from many **shadow releases** (cheap here: our aggregation is numpy, not model training) and tests with the per-target likelihood ratio. We report **ROC-AUC and the low-FPR TPR** (the operationally meaningful metric that AUC-only proxies hide), plus a **reconstruction-decode extraction** attack (MEXTRA analog): decode each released centroid to its nearest note and score a hit when the top-1 decoded note is a true in-bucket member. Fits use a shadow split disjoint from the evaluation shadows (no train-on-test bias).

The clean release is far more leaky than the proxy revealed, and DP still defeats the strong attack (LongMemEval oracle, d=32, 3 seeds × 48 shadow releases):

K	release	AUC	TPR@1%FPR	TPR@0.1%FPR	decode-extract
1024	clean	0.994	0.898	0.778	0.854
1024	$\epsilon \approx 9.3$	0.534	0.011	0.001	0.078
1024	$\epsilon \approx 3.2$	0.505	0.011	0.001	0.018

Calibration exposes what cosine-AUC missed: on the clean pool an adversary re-identifies members at **$\approx 90\%$ true-positive rate at a 1% false-positive budget** and reconstructs the correct in-bucket note for **85% of centroids**: the untreated shared memory is almost fully de-anonymising. The Skellam release drives the same attack to chance: **TPR@1%FPR 0.90 $\rightarrow$ 0.01** (the FPR floor) and **extraction 0.85 $\rightarrow$ 0.08 $\rightarrow$ $\approx$ 0**. The effect holds across the fidelity sweep: clean leakage rises with **K** exactly as §7.3 predicted, while DP pins every cell at chance:

K	clean TPR@1%FPR	$\epsilon \approx 9.3$ TPR@1%FPR	clean decode	$\epsilon \approx 9.3$ decode
512	0.802	0.012	0.809	0.177
1024	0.898	0.011	0.854	0.078
2048	0.916	0.012	0.895	0.039

On **real all-MiniLM-L6-v2 embeddings** the dimension axis behaves identically: clean TPR@1%FPR climbs **0.91 $\rightarrow$ 0.98** and decode **0.80 $\rightarrow$ 0.97** from $d=32 \rightarrow 384$ (higher-**d** memory is more re-identifiable), and DP collapses both to chance at $\epsilon \approx 9.3$, the §7.2 crossover, now with a calibrated attack. (We headline the low-FPR TPR because AUC becomes an unstable estimator at the largest σ , where it can tick up to ≈ 0.59 even as TPR@1%FPR stays at the $\approx 1\%$ floor; the attack is defeated operationally regardless.) This **upgrades the endpoint from a proxy to a calibrated attack**, leaving only the *LLM-agent-prompting* form of MEXTRA/MRMMIA (a live agent querying a text store, a different release model than our centroid pool) as future work.

7.9 The published attacks against a live agent (MEXTRA / MRMMIA)

The attacks so far operate on the released embeddings. To close the loop on the *published* threat model (a live LLM agent that stores and serves memory as **text**), we build an A-MEM/Mem0-style shared-memory agent (local **qwen2.5:7b**) and run both attacks end-to-end. The agent retrieves the top-**k** shared-memory notes for a query into its context and answers; the adversary issues (a) a **MEXTRA** extraction prompt that asks the agent to reproduce its memory verbatim, and (b) a **MRMMIA** membership probe for a candidate note. We hold the distilled notes fixed and swap only the shared-memory back-end:

shared memory back-end	MEXTRA verbatim-recovery	MRMMIA-AUC
raw text (baseline, no privacy)	1.000	0.938
our SecAgg+Skellam centroid pool ($\epsilon \approx 9.3$)	0.000	0.454

(60 users, 883 distilled notes, $K=256$, $d=32$; 40 extraction and 120 membership trials.) On the raw-text agent both published attacks succeed decisively: every targeted member note is reproduced verbatim and membership is inferred at AUC 0.94. Against our release they collapse to **nothing**: the centroid pool contains **no note text**, so the extraction prompt has nothing to surface (the most an adversary can recover is a *public* note decoded from a centroid, §7.8) and the membership probe is at chance. The mechanism's role is thus concrete on the real payload: it does not merely lower an attack score, it **removes the very artifact the published attacks operate on**. (This is a demonstration at modest scale, not a tuned attack sweep; a stronger prompt-injection adversary against the raw baseline would only widen the gap, since our release exposes no text either way.)

7.10 Occupancy: what the count channel actually buys (§5.3)

An empty bucket is the one place the count channel is not redundant: `normalize` turns a pure-noise `S[k]` into a unit vector that competes for top-`k` slots against genuine centroids. We measure (a) how often empty buckets occur, and (b) whether they can be detected *privately*. Real LongMemEval-oracle (10,957 notes, 500 users, $d=32$), $\sigma_v = 0.606$ ($\epsilon \approx 9.28$); occupancy over 5 anchor seeds, detection on a single release. Reproduce with `scripts/agentmem/_occupancy_channel.py`.

(a) At the operating points, there are none.

K	32	64	128	256	512	1024	2048
empty buckets (%)	0.00	0.00	0.00	0.08 ± 0.16	0.74 ± 0.62	6.54 ± 0.97	19.87 ± 0.63

Every retrieval number in this paper is reported at $K \leq 256$, where at most **one bucket of 256** is ever empty; at the recommended $K = 32$ there are none on any seed. The noisy-normalisation failure mode is **absent from the reported utility** rather than suppressed by tuning.

(b) Where it does occur, occupancy is expensive to release. Detection AUC (empty vs occupied; 0.5 = undetectable), against the total ϵ of the release:

occupancy rule	ϵ	K=512	K=1024	K=2048
<code> S[k] </code> vs analytic noise floor <code>$\sigma \cdot C \cdot \sqrt{d}$</code> (free: post-processing)	9.28	0.673	0.613	0.510
noisy count channel, $\sigma_c = 2$ (cheap second composition)	9.78	0.657	0.658	0.579
noisy count channel, $\sigma_c = \sigma_v$ (full second composition)	13.93	0.848	0.801	0.698

Two readings, both load-bearing. First, under **user-level** DP the count channel is *not* the cheap sensitivity-1 object it is under note-level DP: a user contributes up to `n_u` notes, so its clipped count vector carries `C_c  $\approx$  12–20`, and paying a full second composition (ϵ 9.28 $\rightarrow$ 13.93) still leaves detection at AUC 0.70 exactly where it is most needed ($K = 2048$, 20% empty). Second, the free rule — thresholding the released vector's own norm against `$\sigma \cdot C \cdot \sqrt{d}$` , which costs no privacy because `S` is already public — degrades to chance (0.510) in that same regime.

Neither is an implementation shortfall. The DP noise that drives membership inference to chance in a one-contributor bucket (§7.3) is *the same noise* that hides whether the bucket has a contributor at all: **occupancy and membership are the same signal**, and a mechanism cannot hide the member while revealing the bucket. The design consequence is to operate at dense K , where retention is highest anyway (§7.1), rather than to buy a count channel that cannot pay for itself.

8. Discussion and Limitations

The leakage-drop headline is a sparse-regime property, stated honestly. The dramatic 0.9→0.5 tail-AUC drop lives in the **low-count tail**; bulk-bucket AUC is only 0.53–0.59 pre-DP because averaging already protects dense buckets. Density governs both axes: a very dense, coarse- K regime makes DP nearly free **and** has little low-count tail to begin with (so a smaller headline drop). We therefore frame the result as "*DP provably protects the vulnerable tail that a sum mechanism most exposes*," not as an unconditional leakage collapse. The at-scale s -variant (§7.7) **confirms this directly**: with 246k notes it sits in the dense regime, where the low-count tail is **empty** and clean membership-inference AUC is **already ≈0.50**, so DP is nearly free on utility (97–100% retention) and there is little tail leakage left to drop, reported as such, not overclaimed.

Attack strength. We evaluate at three escalating strengths and the endpoint holds at each: (i) an embedding-similarity proxy (§7.2–7.4); (ii) a **calibrated offline-LiRA** MIA reported at low-FPR TPR plus a reconstruction-decode extraction attack (§7.8), which reveal the clean pool is *more* leaky than the proxy showed (TPR@1%FPR ≈ 0.90, ≈85% note reconstruction), yet still fall to chance under DP; and (iii) the **published MEXTRA/MRMMIA attacks against a live agent** (§7.9), which fully extract a raw-text memory (recovery 1.0, AUC 0.94) but recover nothing from the centroid release. The endpoint thus **strengthens** as the attack gets stronger. Limits of (iii): it is a modest-scale demonstration on one open model with untuned extraction prompts; a stronger prompt-injection adversary would widen, not close, the gap, since our release exposes no text regardless. A user-level (rather than note-level) live-agent attack is left to future work.

Scope. Utility is retrieval fidelity of the pooled memory (bucket routing), not end-to-end downstream QA; extending to generated-answer quality is future work. Retrieval is bucket-granular, not exact-note ranking. K / d / clip bounds are set from data percentiles, not tuned per user.

Deployment gaps we specify but do not measure. Three, stated plainly. (i) **Client dropout**: our runs assume full participation. A naive μ/N noise share loses the guarantee precisely when turnout falls — at 50% survival the true budget is $\epsilon \approx 13.9$, not the nominal 9.3 — so a deployment must use the $M_{\min}$ -calibrated share or fault-tolerant noise reconstruction of \$5.6. We specify both; we measure neither. (ii) **Occupancy at high K** : the released leakage code suppresses zero-count buckets using clean counts — a non-private oracle step. It is a no-op at every K where we report retrieval (§7.10a) and cannot affect members (a member's own note occupies its bucket); it does zero the centroid for the 1.2% / 3.6% of non-members that fall in an unpooled bucket at $K = 1024 / 2048$, the settings used only for the leakage tables of §7.3, shifting clean and private tail-AUC alike. §7.10b prices the private replacements, none of which is simultaneously free and accurate; the honest reading is that near-empty

buckets cannot be *both* privately aggregated and privately enumerated. (iii) **δ at production scale:** $\delta = 10^{-5}$ is 200 $\times$ below $1/N$ at our $N = 500$ **users**, but a 10^5 -user fleet must retighten it (\$5.4), at a cost of $< 23\%$ ϵ .

Non-novel components, explicitly. The dimension dependence of DP is classical [19, 20]; we claim only its *coupled* manifestation across leakage and utility in agent memory. The SecAgg + Skellam path is prior work [2, 5, 6], and the LSH-bucketing-plus-DP skeleton overlaps the centralized *DP Datastore Generation* [15] (§2); the novelty is the federated/curator-free SecAgg+Skellam composition, the agent-memory payload, and the measured attack-drop endpoint.

9. Conclusion

Shared LLM-agent memory can be made **useful and provably private simultaneously**. By aggregating per-user bucketed memory embeddings under Secure Aggregation and the Skellam mechanism, the pooled memory carries a curator-free central-DP guarantee, retains **95–96% of clean retrieval utility at $\epsilon \approx 9.3$** in the tiny-d / coarse-K regime agent payloads occupy, and drives worst-case membership inference on vulnerable members from **near-certain to chance**. The dimension/density crossover makes tiny-d a joint optimum (a regulariser that lowers pre-DP leakage and raises post-DP retention with one knob), the discrete mechanism is privacy-free at our resolution, a vector-only release dominates once bucket occupancy is accounted for, and, reassuringly, the realistic distilled payload, which leaks *more* in the clear, is protected *more* decisively by DP. Across three escalating attack strengths (an embedding proxy, a calibrated offline-LiRA with reconstruction-decode extraction, and the published MEXTRA/MRMMIA attacks against a live agent), the endpoint holds and in fact strengthens: the raw-text memory is fully extractable while our release exposes no note text at all. The at-scale run confirms the honest boundary of the headline rather than overturning it.

10. Reproducibility

Active code lives in `scripts/agentmem/` (see `scripts/README.md`); the pooled crypto is `qpriviot_fl/privacy_utils.py`.

- **Utility:** `_longmemeval_probe.py` (real), `_agentmem_probe.py` (controls), `_longmemeval_distilled_utility.py` (distilled).
- **Leakage (proxy):** `_longmemeval_leakage.py`, `_agentmem_leakage.py`, `_longmemeval_distilled_analysis.py` (raw-vs-distilled).
- **Leakage (calibrated, §7.8):** `_agentmem_lira.py`, offline-LiRA MIA (AUC + low-FPR TPR) and reconstruction-decode extraction, via shadow releases.
- **Leakage (live agent, §7.9):** `_agentmem_llm_attack.py`, an A-MEM/Mem0-style shared-memory agent (local Ollama) under end-to-end MEXTRA extraction + MRMMIA membership prompts, contrasting a raw-text memory vs our centroid release.
- **Accounting:** `_skellam_accounting.py` (Skellam-RDP ϵ ; discretisation-free check).

- **Distillation:** `_longmemeval_distill.py` turns LongMemEval turns into A-MEM/Mem0-style notes via a local Ollama model (`qwen2.5:7b`); it checkpoints incrementally.
- **Drivers → tables/figures:** `scripts/shell/rerun_grid.sh` runs the 33-config, 5-seed grid into `experiment_results/rerun_grid/*.json`; `scripts/shell/rerun_svariant.sh` runs the at-scale `s`-variant (§7.7) into `experiment_results/rerun_grid_s/*.json`; `scripts/shell/rerun_lira.sh` runs the calibrated-attack grid (§7.8) into `experiment_results/lira/*.json`; `rerun_tables.py` and `rerun_figures.py` regenerate `SUMMARY_TABLE.md` and `figures/fig_*`. Each script has an additive `--json dump`; the crypto path is byte-identical to the released `privacy_utils`.

Run: `bash scripts/shell/rerun_grid.sh && python scripts/agentmem/rerun_tables.py && python scripts/agentmem/rerun_figures.py` (and `bash scripts/shell/rerun_svariant.sh` for §7.7, `bash scripts/shell/rerun_lira.sh` for §7.8; §7.9 needs a local `ollama serve` with `qwen2.5:7b`).

References

Note: the 2025–2026 agent-memory arXiv identifiers below were verified against the live arXiv/venue record (2026-07); titles and attributions match the published metadata.

- [1] McMahan et al. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS 2017. [2] Bonawitz et al. *Practical Secure Aggregation for Privacy-Preserving Machine Learning*. CCS 2017. [3] Abadi et al. *Deep Learning with Differential Privacy*. CCS 2016. [4] Mironov. *Rényi Differential Privacy*. CSF 2017. [5] Agarwal, Kairouz, Liu. *The Skellam Mechanism for Differentially Private Federated Learning*. NeurIPS 2021 (arXiv:2110.04995). [6] Kairouz et al. *The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation*. ICML 2021. [7] Reimers, Gurevych. *Sentence-BERT*. EMNLP 2019 (all-MiniLM-L6-v2). [8] Wu et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. 2024. [9] Xu et al. *A-MEM: Agentic Memory for LLM Agents*. 2024. [10] Chhikara et al. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. 2024. [11] Wang et al. *Unveiling Privacy Risks in LLM Agent Memory*. ACL 2025 (arXiv:2502.13172); introduces the **MEXTRA** memory-extraction attack. [12] Chen, Pang, Wang. *MRMMIA: Membership Inference Attacks on Memory in Chat Agents*. arXiv:2605.27825. [13] Rezazadeh et al. *Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control*. arXiv:2505.18279. [14] Chen et al. *MemPrivacy: Privacy-Preserving Personalized Memory Management for Edge-Cloud Agents*. arXiv:2605.09530. [15] Abouelenein, Torki. *Differentially Private Datastore Generation for Retrieval-Augmented Inference*. arXiv:2606.01413. [16] Hou et al. *POPri: Private Federated Learning using Preference-Optimized Synthetic Data*. arXiv:2504.16438. [17] Chen et al. *Fed-SE: Federated Self-Evolution for Privacy-Constrained Multi-Environment LLM Agents*. arXiv:2512.08870. [18] Lin et al. *A Survey on the Security of Long-Term Memory in LLM Agents*. arXiv:2604.16548. [19] Chen et al. *The Effect of Dimensionality on Differentially Private Deep Learning*. ICML 2022 (arXiv:2203.03761). [20] Bassily, Smith, Thakurta. *Private Empirical Risk Minimization*. FOCS 2014.